

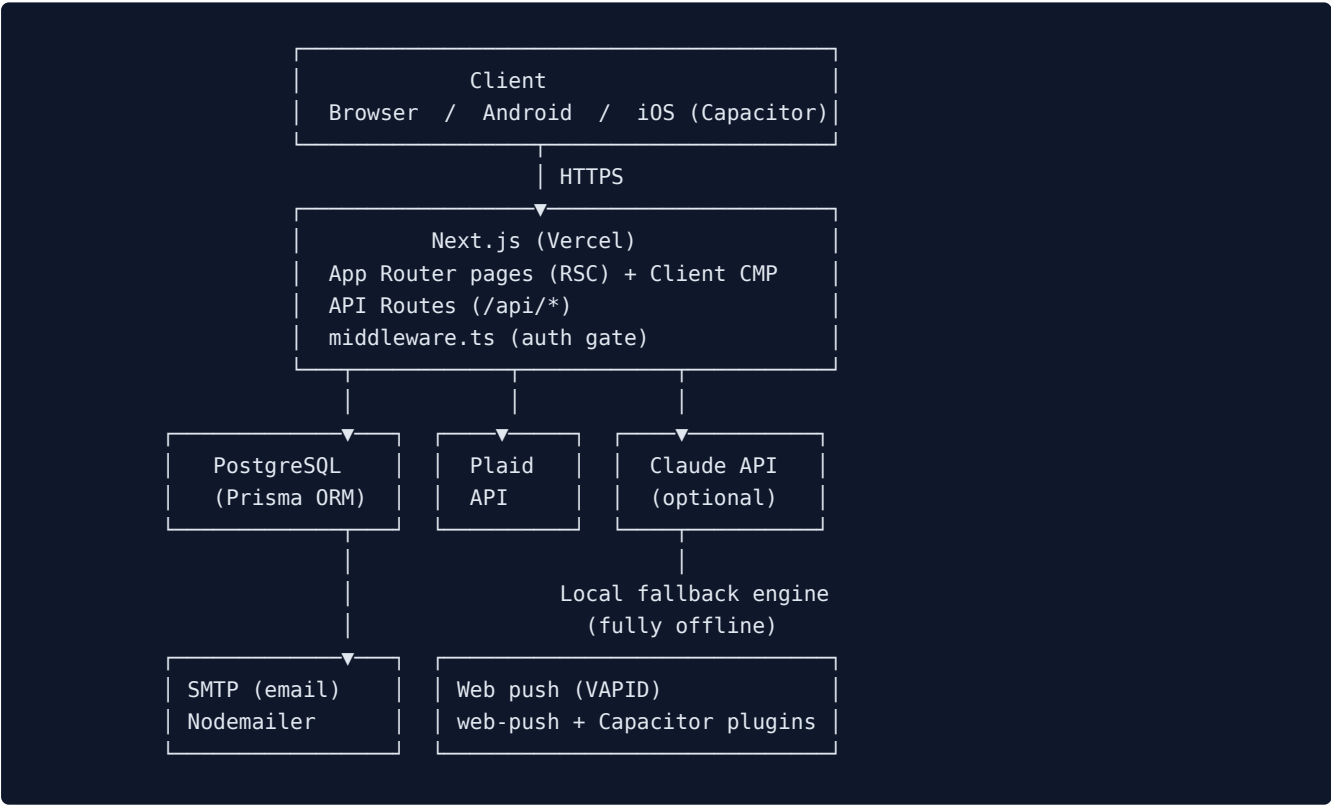
FinTrack Technical Documentation

Version: 1.0
Audience: Engineering and operations
Related documents: `README.md` (orientation), `FinTrack-User-Manual.pdf` (product behaviour), `PRODUCTION_READINESS.md` (launch readiness)

1. System overview

FinTrack is a server-rendered web application with client-side interactive surfaces, wrapped for Android and iOS with Capacitor. The server runs as Next.js serverless functions on Vercel and communicates with a PostgreSQL database, the Plaid bank-connectivity API, and the Anthropic Claude API.

1.1 System context diagram



2. Technology stack

Layer	Technology	Notes
-------	------------	-------

Framework	Next.js 15 (App Router), React 18, TypeScript 5	Server components for data-heavy pages
Styling	Tailwind CSS 3.4, custom CSS variables	Dark and light themes, class-based
Charts	Recharts 2.12	Area and pie charts for cashflow and categories
Component primitives	Radix UI	Dialog, Dropdown, Select, Tooltip
Database	PostgreSQL	Managed via Prisma 5 ORM
ORM	Prisma	29 models; see section 5
Auth	NextAuth 4 (JWT strategy)	Credentials + Google + GitHub
Bank connectivity	Plaid (<code>plaid</code> SDK 42.x)	<code>transactionsSync</code> cursor-based import
AI	<code>@anthropic-ai/sdk</code>	Claude for analysis and chat; local fallback
Validation	Zod 3	All API request bodies
Email	Nodemailer	SMTP; verification and password reset
Push	<code>web-push</code> + Capacitor Push	VAPID for web; native push via Capacitor
Mobile	Capacitor 6	<code>webDir: public</code> , live server URL
Deployment	Vercel	<code>vercel.json</code> build configuration

3. Repository structure

```
src/
├── app/
│   ├── page.tsx           Public landing page
│   ├── layout.tsx         Root layout and fonts
│   ├── globals.css        Design tokens, utilities, animations
│   ├── providers.tsx      Client providers (theme)
│   ├── middleware.ts      Auth and email-verification gate
│   ├── auth/              Login, register, verify, reset, forgot
│   ├── onboarding/        Post-registration bank linking
│   ├── dashboard/         Page.tsx (server) for every feature screen
│   └── api/               Route handlers; see section 8
├── components/
│   ├── accounts/          MobileAccounts
│   ├── bank/              LinkBankButton (Plaid Link)
│   ├── charts/            CashflowRibbon, CategoryChart
│   ├── chat/              ChatWidget (floating assistant)
│   ├── dashboard/         MobileDashboard, AccountSwitcher
│   ├── layout/            DashboardShell, Sidebar, TopBar, MobileNav, GradientHeader
│   ├── notifications/     NotificationDropdown
│   ├── providers/         ThemeProvider
│   ├── settings/          Profile, Security, Privacy, Preferences, etc.
│   ├── transactions/      MobileTransactions
│   └── ui/                BottomSheet, StatCard, SyncButton, TransactionRow, Disclosure
├── hooks/                 useCurrency, useHaptics, usePullToRefresh, usePushNotifications,
├── useProfile
├── lib/
│   ├── ai.ts              Claude analysis engine + chat system
│   ├── auth.ts            NextAuth configuration
│   ├── plaid.ts           Plaid API client
│   ├── encryption.ts      AES-256-GCM + webhook HMAC verification
│   ├── prisma.ts          Prisma client singleton
│   ├── currency.ts        Formatting and amount splitting
│   ├── accountTheme.ts    Brand colour system for mobile headers
│   ├── email.ts           SMTP senders
│   ├── notifications.ts   Push notification sender
│   ├── push-notifications.ts Web push helpers
│   ├── capacitor-push.ts  Capacitor push setup
│   ├── categorize.ts      AI categorisation + rule engine
│   └── geocode.ts / location.ts / prices.ts / haversine.ts  Location and merchant data
├── types/
│   ├── index.ts           Shared interfaces
│   └── next-auth.d.ts     Session type augmentation
├── prisma/
│   └── schema.prisma      Database schema
├── android/               Capacitor Android project
├── docs/                  Documentation
└── .github/workflows/build.yml  APK and IPA CI builds
```

4. Frontend architecture

4.1 Rendering model

- **Server components** (`app/dashboard/page.tsx` and peers) fetch session data and page data with Prisma, then render. Examples: `DashboardPage`, `accounts`, `budgets`, `goals`.
- **Client components** manage interactive state and call `/api/*`. Examples: `MobileDashboard`, `MobileTransactions`, `chat`, `intelligence`.
- The dashboard page passes a serialisable `stats` object into `DashboardClient`, which switches between `DashboardClient` (desktop) and `MobileDashboard` (under `lg:` breakpoints).

4.2 State and data flow

Data flows into the dashboard server-first, then interactively through API routes:

- Server components read the session via `getServerSession(authOptions)` and query Prisma directly.
- Interactive actions call `fetch('/api/...')` with JSON bodies.
- After mutations, client components call `router.refresh()` to re-run server components and pick up fresh data.

4.3 Mobile experience

- `MobileDashboard` implements the home screen: pull-to-refresh (custom `usePullToRefresh` hook with spring animation), skeleton loading, snap-scroll account selection, animated balance display, and a quick-add transaction bottom sheet.
- `BottomSheet` is Radix Dialog-based with drag-to-dismiss.
- `useHaptics` wraps Capacitor Haptics with a web fallback (no-op).
- `MobileNav` provides the five-tab bottom navigation on viewports below `lg`.

4.4 Design system and theming

- **Tokens.** CSS variables in `globals.css`: `--accent`, `--ink`, `--surface`, `--surface-muted`, `--line`, `--wash-base`. The `accent` is a refined bank blue.
- **Components.** Tailwind component classes (`card`, `btn-primary`, `btn-secondary`, `btn-accent`, `input`, `label`, `badge`, `section-title`) in `@layer components`.

- **Mobile brand headers.** `GradientHeader` renders a full-width header on mobile using the FinTrack signature gradient (cyan to violet, matching the FinTrack mark) with two slowly drifting aurora orbs (`ft-orb` , `ft-orb-a` , `ft-orb-b` in `globals.css`). Colour is driven per account by `lib/accountTheme.ts` .
- **Reduced motion.** A global `prefers-reduced-motion` media query disables animation and transitions.

4.5 Currency handling

`lib/currency.ts` provides locale-aware formatting. `splitAmount` decomposes a value into sign, integer, decimal, and cents for the large split balance display. `useCurrency` reads the user's currency preference and exchanges display values when the account currency differs using cached rates from `/api/rates` .

5. Data model

The Prisma schema defines 29 models. The principal entities:

Model	Purpose	Key fields
<code>User</code>	Account	email (unique), password hash, emailVerified, currency, location
<code>Account</code> / <code>Session</code> / <code>VerificationToken</code>	NextAuth provider accounts and sessions	provider, tokens
<code>LinkedBank</code>	Plaid connection	institution, account type/name, encrypted token, lastSynced
<code>Transaction</code>	Imported transactions	amount, direction, description, merchant, category, status, runningBalance; index on [userId, date]
<code>ManualTransaction</code>	User-entered transactions	amount, direction, description, optional receipt
<code>Receipt</code>	Receipt blobs	filename, data (Bytes), mimeType
<code>Budget</code>	Category limits	category, amount, period; unique [userId, category, period]
<code>Goal</code>	Savings goals	name, targetAmount, currentAmount, deadline, color
<code>Bill</code>	Recurring bills	amount, dueDate (day), frequency, reminderDays
<code>DebtPlan</code> / <code>Liability</code>	Debt strategies and debts	strategy, interestRate, minPayment

ChatMessage	Assistant history	role, content
Alert	System alerts	type, severity, read, data (JSON)
AiInsight	Cached analysis	type, period, data (JSON), expiresAt
CategorizationRule	Auto-categorisation rules	matchType, matchValue, category, priority
Asset / Liability / Investment / CreditScore	Net worth and investments	value, shares, score
TaxEntry	Tax records	year, type, amount
Household / HouseholdMember	Shared households	role, joinedAt
Notification / PushSubscription	Notifications and push	type, read; endpoint (unique), p256dh, auth
Vault / RoundUpRule	Savings pots and round-ups	targetAmount, currentAmount, isActive

5.1 Schema naming

Plaid-era field names (`plaidAccountId` , `accessToken` , `plaidTransactionId`) are used throughout. The earlier Teller naming was renamed during pre-launch cleanup; no legacy columns remain in the schema or code paths.

6. Authentication and authorisation

- **Providers.** Credentials (email/password with bcrypt, cost factor 12), Google OAuth, and GitHub OAuth. OAuth providers are registered only when their environment variables are present.
- **Sessions.** JWT strategy with the session stored in a signed, HttpOnly cookie. The `jwt` callback re-reads `emailVerified` from the database on each token refresh so account state stays current.
- **Email verification.** `middleware.ts` redirects unauthenticated requests to `/auth/login` and unverified sessions to `/auth/verify` for `/dashboard/*` and `/onboarding` .
- **Verification flow.** Registration creates the user with a verification token; `sendVerificationEmail` mails a 24-hour link. The link handler verifies and sets `emailVerified` .
- **Password reset.** A token-based reset link (1-hour expiry) via `sendPasswordResetEmail` .
- **API protection.** Individual route handlers validate the session with `getServerSession(authOptions)` and scope all queries by

`session.user.id` . Multi-tenant data isolation depends on this user-scoping; no cross-user route was found during review.

7. Bank connectivity (Plaid)

7.1 Link flow

1. `GET /api/plaid/link-token` creates a Link token for the signed-in user (`client_user_id` = user id) with the `Transactions` product and US/EU country codes.
2. The client opens Plaid Link via `react-plaid-link` (`LinkBankButton`).
3. The user authenticates with their bank. Plaid returns a public token.
4. `POST /api/plaid/exchange` swaps the public token for an access token and item id, encrypts the access token (AES-256-GCM), and stores a `LinkedBank` row.
5. `POST /api/plaid/sync` imports transactions (below).

7.2 Sync flow

For each linked bank:

1. Decrypt the access token.
2. Call `transactionsSync` with the stored cursor, up to 500 transactions per call, following `has_more` .
3. Upsert `added` and `modified` transactions, delete `removed` ones, keyed by `plaidTransactionId` (Plaid transaction id).
4. Persist `next_cursor` and `lastSynced` .

7.3 Security

- Access tokens are encrypted at rest and decrypted only inside server functions at sync time.
- The encryption key is validated to be 32 bytes at module load (`lib/encryption.ts`).
- Webhook signatures are verified with an HMAC helper (`verifyWebhookSignature`); Plaid webhooks are not yet wired (see production readiness document).

8. API reference

All routes live under `/api` . Unless noted, every route requires a session and scopes data to the authenticated user. Request bodies are validated with Zod.

8.1 Auth

Method	Route	Description
POST	/api/auth/register	Create account, send verification email
POST	/api/auth/verify-email	Verify email token
POST	/api/auth/forgot-password	Send reset email
POST	/api/auth/reset-password	Set a new password
POST	/api/auth/change-password	Change password (requires current password)
POST	/api/auth/change-email	Change email, send confirmation
POST	/api/auth/confirm-email	Confirm a changed email
GET/POST	/api/auth/[...nextauth]	NextAuth handler
GET	/api/accounts/me	Current user profile
POST	/api/auth/profile	Update profile fields

8.2 Bank connectivity

Method	Route	Description
GET	/api/plaid/link-token	Create a Plaid Link token
POST	/api/plaid/exchange	Exchange public token, store encrypted access token
GET	/api/plaid/accounts	List linked banks with balances
GET	/api/plaid/accounts/[id]	Single linked bank
DELETE	/api/plaid/accounts/[id]	Unlink account (calls Plaid <code>item/remove</code>)
POST	/api/plaid/sync	Import transactions for all linked banks

8.3 Transactions

Method	Route	Description
GET	/api/transactions	List/filter transactions (search, category, date range)
GET	/api/transactions/search	Search endpoint
POST	/api/manual-transactions	Create a manual transaction
GET	/api/manual-transactions	List manual transactions
POST	/api/categorize	AI categorisation of a transaction
GET/POST	/api/rules	Categorisation rules CRUD
POST	/api/rates	Currency conversion rates (cached)

8.4 Planning features

Method	Route	Description
GET/POST	/api/budgets	List and create budgets
GET/POST	/api/goals	List and create goals; update progress
GET/POST	/api/bills	List and create bills
GET	/api/bills/upcoming	Upcoming bills within reminder window
GET/POST	/api/debt-plans	Debt liabilities and plans
GET/POST	/api/vault	Vault pots CRUD
POST	/api/vault/round-up	Apply round-up to a pot
GET/POST	/api/subscriptions	Recurring subscription detection

8.5 Net worth and finance

Method	Route	Description
GET/POST	/api/assets	Asset CRUD
GET/POST	/api/liabilities	Liability CRUD
GET	/api/net-worth	Computed net worth
GET/POST	/api/investments	Investment CRUD
GET/POST	/api/credit-score	Credit score log
GET/POST	/api/tax-entries	Tax entries by year
GET/POST	/api/households	Households and members
GET	/api/health-score	Financial health calculation

8.6 Intelligence and chat

Method	Route	Description
POST	/api/intelligence	Run AI analysis; reads/writes <code>AiInsight</code> cache (1-hour TTL)
GET/POST	/api/chat	Chat history and message exchange
GET	/api/location	Reverse geocoding and location analysis
GET	/api/prices/search	Local price comparison
GET	/api/alerts/generate	Generate alerts from current data

8.7 Notifications and reporting

Method	Route	Description
GET/POST	<code>/api/notifications</code>	In-app notifications
GET	<code>/api/push/public-key</code>	VAPID public key for web push
POST	<code>/api/push/register</code>	Register a push subscription
GET	<code>/api/reports</code>	Monthly report data

9. AI engine

9.1 Analysis (`lib/ai.ts` , `analyzeSpending`)

Computes fifteen analysis dimensions from transaction data: summary, top insight, category breakdown with trends, merchant alternatives (via Overpass API or generic fallback), location insights, savings opportunities, cashflow health rating, subscription overlaps, recurring price trends, what-if scenarios, a 14-day cashflow forecast, spending patterns, merchant concentration risk, hidden recurring charges, and a sustainability score.

Results are cached in `AiInsight` with a 1-hour expiry keyed by user, type, and period.

9.2 Chat (`lib/ai.ts` , `chatWithData`)

- **Primary path.** Claude with function-calling tools: `search_transactions` , `get_category_totals` , `get_merchant_info` .
- **Fallback path.** A deterministic local engine that computes aggregates and matches intent through pattern matching, supporting 15+ question types and time-range filtering. This path requires no API key and keeps the assistant operational during provider outages.
- **Context.** `/api/chat` loads the last 20 messages and up to 500 recent transactions plus budgets, goals, and debts before calling the engine.

10. Security

- **At rest.** Passwords bcrypt cost 12; bank tokens AES-256-GCM with per-message random IV and authentication tag; encryption key length enforced.
- **In transit.** All traffic HTTPS (Capacitor `cleartext: false`); production behind Vercel.
- **Sessions.** Signed JWT in HttpOnly cookie; CSRF protection via NextAuth; email-verification gate in middleware.
- **Input.** Zod schemas on API bodies; no raw SQL.

- **Rate limiting.** In-memory limiter (`lib/rate-limit.ts`) on registration, password reset, email verification/change, password change, and the AI chat and intelligence endpoints. Back with an external store for multi-region deployments.
- **Headers.** CSP, HSTS, X-Content-Type-Options, X-Frame-Options, Referrer-Policy, and Permissions-Policy set in `next.config.js` .
- **Bank access.** Read-only via Plaid; tokens never serialised to the client; no write/transfer capability exists in the integration.
- **Gaps tracked.** Audit log, secrets rotation, and session revocation are outstanding (see `PRODUCTION_READINESS.md`).

11. Mobile builds (Capacitor)

`capacitor.config.ts` :

- App id `com.fintrack.app` , web directory `public` , live server URL `https://fintrack-pearl-eight.vercel.app` .
- Android scheme `https` with cleartext disabled.
- Android release builds require a keystore (currently undefined in config; CI builds debug APKs).
- iOS scheme `App` .

Build scripts in `package.json` :

Script	Purpose
<code>npm run cap:sync</code>	Sync Capacitor projects
<code>npm run cap:android</code>	Open Android project
<code>npm run apk</code>	Sync + assemble debug APK
<code>npm run apk:release</code>	Sync + assemble release APK (requires signing)
<code>npm run ipa</code>	Build and export iOS archive (macOS + Xcode)
<code>npm run vapid:generate</code>	Generate web-push VAPID keys

The CI workflow `.github/workflows/build.yml` builds the Next.js app, syncs Capacitor, and produces a debug APK on Ubuntu and an unsigned iOS build on macOS.

12. Deployment

- **Hosting:** Vercel. `vercel.json` sets the build command to `prisma generate && next build` .

- **Database:** External PostgreSQL (the app is database-host agnostic; Neon, Supabase, or RDS all work).
- **Environment:** All secrets in `process.env` (see `.env.example`).
`NEXT_PUBLIC_*` variables are exposed to the client; all others are server-only.
- **Cron:** No production schedulers are configured yet. Recurring work (daily sync, bill reminders, alerts) is invoked from request paths today.

13. Observability (current state)

- **Errors:** Unhandled errors surface in Next.js error boundaries; no third-party error tracking is connected.
- **Logs:** `console` output is available in Vercel function logs.
- **Alerting:** None configured. Availability, sync health, and AI fallback usage are not instrumented.
- **Required additions** (costed in `PRODUCTION_READINESS.md`): Sentry, uptime checks, structured logs, SLOs, and scheduled-job infrastructure.

14. Local development

Prerequisites: Node.js 20+, PostgreSQL, and the environment variables from `.env.example` .

```
npm install
cp .env.example .env.local # then fill in values
npm run db:push
npm run db:generate
npm run dev
```

Validation:

```
npx tsc --noEmit # type checking
npm run build # production build
```

There is no automated test suite yet; the production readiness document schedules one before launch.

15. Known limitations

1. No automated test suite or CI test stage.
2. Plaid webhooks (sync, bill reminders, alerts) run on request paths rather than scheduled jobs.

3. Credit score and investment values are manually entered; no live market or bureau feeds.
4. AI is uncapped per user, relying on a 1-hour cache to control cost.
5. In-memory rate limiting (single-instance) rather than an external store; no audit log.
6. Mobile release builds are unsigned in the default configuration.
7. Receipts are stored as database blobs rather than object storage.